

Module 10 — Stretch: Auth Layer on the FastAPI Service

Honors Track. This is an Honors Track stretch assignment — not required for program completion, but required for Honors distinction. Stretch is for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently. If you are unsure whether stretch is right for you this week, talk to your TA before starting.

Due: Tuesday end of day. Resubmissions are accepted through Saturday of the assignment week.

Prerequisite: Module 10 Applied Lab. This stretch builds on the Lab repo (`m10-l10`) directly — no new template.

Objective

Add a two-tier authentication layer to the Lab's FastAPI backend: (a) **API-key authentication** (header `X-API-Key`) for service-to-service callers, and (b) **JWT bearer authentication** (header `Authorization: Bearer <token>`) for user-facing calls. Extend the Next.js frontend with a minimal login page that obtains a JWT (JSON Web Token) from a new `/auth/login` endpoint and includes it on subsequent calls.

You are turning the Lab's open endpoints into endpoints with an access policy — the same discipline you would apply to any production service shipping to real callers.

Setup

Branch off your completed Lab on a new branch:

```
cd m10-l10
git checkout main
git pull
git checkout -b stretch-tue-auth
pip install 'python-jose[cryptography]>=3,<4' 'passlib[bcrypt]>=1.7,<2'
# Add BOTH lines to api/requirements.txt so the autograder workflow
# installs them in CI:
#   python-jose[cryptography]>=3,<4
#   passlib[bcrypt]>=1.7,<2
```

Also create these env vars locally — they are read by `api/auth.py` and `api/main.py` (Task 1 + Task 2). Add the same keys (with placeholder values) to `api/.env.example` and commit the example; never commit the real `.env`.

```
# api/.env.example - append:
JWT_SECRET=<long-random-string>      # used by api/auth.py for HS256 signing
JWT_ALGORITHM=HS256
API_KEY_VALID=<dev-api-key>          # accepted X-API-Key value
```

If you have not already installed the Module 10 packages, go back to the [Setup: install before the concept session](#) section of the reading.

Learning outcomes

By completing this stretch you will be able to:

- Use FastAPI's `Security`, `APIKeyHeader`, and `OAuth2PasswordBearer` (OAuth2 = Open Authorization 2.0) primitives to declare auth requirements as `Depends()` chains.
- Issue and verify JWTs with `python-jose`, including expiry handling.
- Express different auth requirements per endpoint — `/extract` requires API-key OR JWT, `/admin/...` requires JWT only.
- Distinguish HTTP 401 (no or invalid credential) from HTTP 403 (valid credential, wrong scope) and surface both distinctly on the frontend.

Tasks

1. `api/auth.py` — credential verifiers

Author three functions:

- `verify_api_key(api_key: str = Security(api_key_header)) -> str` — verifies the `X-API-Key` header against `os.getenv("API_KEY_VALID")`; raises 401 if missing or invalid; returns the key on success.
- `verify_jwt(token: str = Depends(oauth2_scheme)) -> dict` — verifies the JWT with `python-jose`, returns the decoded payload on success; raises 401 if missing, 401 if signature invalid or expired.
- `create_access_token(subject: str, expires_minutes: int = 60) -> str` — issues a JWT signed with `JWT_SECRET` from env.

The JWT secret must come from env, never hardcoded. The autograder grep-checks for hardcoded secrets.

```
auto_error=False matters for the 401 contract. FastAPI's APIKeyHeader and
OAuth2PasswordBearer default to auto_error=True, which raises an HTTPException with status
403 (not 401) when the header is absent. The autograder expects 401 for both "no credentials" and
"invalid credentials." Construct both schemes with auto_error=False so a missing header returns
None, and have your OR-logic dependency raise 401 explicitly when neither credential validates:

api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/auth/login", auto_error=False)
```

2. `api/main.py` updates

- Add `POST /auth/login` accepting `{username: str, password: str}` and returning `{access_token: str, token_type: "bearer"}`. For the stretch, the user store is a hardcoded dev fixture (one user; password verified with `passlib`'s `bcrypt` hash); the production-grade user store is out of scope.
- Decorate the three Lab endpoints with the **API-key OR JWT** dependency.
- Add a new `GET /admin/echo` endpoint requiring **JWT only** (no API-key fallback). Returns the decoded JWT payload as JSON.

3. `web/pages/login.tsx`

A minimal login form that POSTs `{username, password}` to `/auth/login`, stores the returned `access_token` in `localStorage`, and redirects to `/extract` on success.

```
SSR guard for localStorage. Next.js pre-renders every page on the server during 'npm run build';
localStorage does not exist there. A top-level localStorage.getItem(...) or a
localStorage.setItem(...) in component-body code (outside useEffect / an event handler)
throws ReferenceError: localStorage is not defined and fails the entire production build. Keep
localStorage access inside the form submit handler (onSubmit) or inside a useEffect block —
both run only in the browser. The authFetch wrapper below already uses the typeof window !==
"undefined" guard for the same reason.
```

4. `web/lib/api.ts` updates

A small auth-aware fetch wrapper:

```
export async function authFetch(url: string, init: RequestInit = {}): Promise<Res> {
  const token = typeof window !== "undefined" ? localStorage.getItem("access_token") : null
  const headers = new Headers(init.headers)
  if (token) headers.set("Authorization", `Bearer ${token}`)
  return fetch(url, { ...init, headers })
}
```

The three Lab pages (`extract.tsx`, `kg.tsx`, `rag.tsx`) switch from raw `fetch` to `authFetch` and branch on 401 (redirect to `/login`) and 403 (display an "insufficient scope" message).

Tests

The autograder verifies:

- `/extract` without credentials returns 401.
- `/extract` with an invalid API key returns 401.
- `/extract` with a valid API key returns 200.
- `/extract` with a valid JWT returns 200.
- `/auth/login` with correct credentials returns a JWT; the JWT verifies.
- `/admin/echo` with a valid API key but no JWT returns 403 (the credential is valid for `/extract` but the admin endpoint requires the JWT scope specifically).
- `/admin/echo` with a valid JWT returns 200 with the decoded payload.
- No hardcoded JWT secret in source.
- The token returned by `/auth/login` decodes with the configured `JWT_SECRET` / `JWT_ALGORITHM`, carries an `exp` claim, and authenticates against `/extract`. (A stub like `"access_token": "admin"` will not pass.)

The frontend login flow (login page → submit credentials → land on `/extract` → call succeeds) is reviewed from the screenshot in your submission PR; CI exercises the backend only. The login UI is a required deliverable, not optional — error states (wrong password, expired token, network failure) must render clear messages to the user.

Run locally:

```
pytest tests/auth -v
(cd web && npm run build)
# Optional local Playwright smoke against your own stack — not run by CI:
# npx playwright test tests/frontend/playwright/auth.spec.ts
```

Submission

- Commit your changes to the `stretch-tue-auth` branch and push to your Lab fork.
- Open a Pull Request (PR) within your fork: head = `stretch-tue-auth`, base = `main`. **Do not merge** — the Lab is graded independently from `main`, and the open Lab PR (`lab-10-fastapi-kg-service` → `main`) stays untouched. Because both PRs target `main` separately, their diffs do not interfere as long as `stretch-tue-auth` is branched off `main` (not off your Lab feature branch).
- Confirm the stretch autograder workflow succeeds.

Your PR description must include:

- A one-paragraph summary of your auth design (how API-key OR JWT is expressed; how the user store is structured).
- Confirmation that the autograder workflow passes.
- A screenshot of the login page → `/extract` page flow.
- Paste your PR URL into TalentLMS → Module 10 → Stretch Tue submission to submit this assignment.

